

Requirements Elicitation Frameworks

Requirements elicitation is the disciplined process of discovering what stakeholders need from a solution. It combines research and interviewing to capture requirements that drive process models, system designs, and user stories. For example, to model an *order-to-cash* workflow in manufacturing, a business analyst might interview sales staff, review order forms, and observe the invoicing process to build an accurate process map. Across industries – from **finance** to **manufacturing** to the **public sector** – common techniques such as interviews, workshops, surveys, observation, and document analysis are used. Each technique yields different insights: interviews and workshops generate rich qualitative insights, surveys gather broad quantitative input, observation uncovers real-world practices, and document analysis leverages existing policies or forms. This report compares the major elicitation methods, shows how they feed into structured process models, and outlines ways to ensure the requirements gathered are **clear, testable, atomic, and traceable** ¹ ². We also cover validation practices (reviews, prototypes, acceptance criteria) and role-specific questioning strategies for executives, frontline staff, and technical users. A practical checklist, example Gherkin acceptance criteria, and dialogue snippets illustrate these concepts.

Major Elicitation Techniques

1. Stakeholder Interviews. A structured interview is the most common elicitation method ³. In a one-on-one or small-group interview, the analyst asks targeted questions to stakeholders about their needs and workflows. Interviews are ideal for detailed, context-rich information. For example, interviewing a finance manager about an *order-to-cash* process can reveal the sequence of approval steps, exception paths (e.g. credit holds), and business rules that should go into a workflow diagram. To prepare, an analyst typically creates an agenda and a **question checklist**, covering who, what, where, when, why, and how. (As one practitioner advises, “requirements will answer all the important questions... think about the how, where, when, who, what, and why” ⁴.) During interviews, actively listen and probe on unclear points. Interviews feed directly into process documentation: the BA can translate conversations into use cases, user stories, or process flow drafts. A well-run interview yields actionable notes and often leads to follow-up questions.

2. Facilitated Workshops. Workshops (sometimes called **JAD sessions** or focus groups) bring multiple stakeholders together for a guided session. In a workshop, participants from different roles collaborate—often using whiteboards, flip charts, or models—to sketch out processes or solutions. For example, a cross-functional team might spend a half-day mapping the *new employee onboarding* process: HR leads explain required forms, IT shows system setup steps, and security outlines badge issuance. Workshops encourage dialogue and consensus, and can combine techniques like brainstorming and paper prototyping ⁵. They are especially useful for building shared process diagrams (swimlanes, workflow maps) because everyone can react to and refine the model in real time. According to experts, effective workshop activities include whiteboard “brainstorm” breaks and small-group exercises (e.g. “focus group” discussions) to generate ideas and prioritize requirements ⁵. After the session, the output is a draft process model or feature list that reflects the group’s agreed understanding.

3. Surveys and Questionnaires. Surveys collect information from a larger or dispersed audience. A questionnaire can be sent to many users (e.g. all branch employees in a bank) to gather metrics or opinions about a process. For instance, in an *incident response* process review, an analyst might survey helpdesk staff on common incident types and response times. Surveys are cost-effective for broad input and can surface “anonymous” feedback that might not emerge in person ⁶. However, they tend to yield less depth; responses may require follow-up clarification. Survey data can quantify frequency of events or satisfaction levels, which then inform the process model or highlight pain points to address.

4. Observation (Job Shadowing). In observation, the analyst watches real users perform their daily tasks, documenting what happens and when. This is especially valuable for understanding **current-state processes**. For example, a BA might sit with a customer service rep and watch how a loan application is handled from start to finish. This reveals exact workflow steps, the sequence of screens or forms, and informal workarounds. Observation can be passive (just watching) or interactive (asking questions while the user works) ⁷ ⁸. Even if no notes are taken, watching users can highlight steps that stakeholders assume are “obvious.” Observation findings are directly translated into process diagrams or activity flows, ensuring that the model matches actual practice.

5. Document and Artifact Analysis. Reviewing existing documents, forms, and diagrams can uncover requirements without taking up stakeholder time ⁹. For example, a procurement process may already have SOP manuals, policy documents, or legacy data schemas. An analyst can glean process steps, business rules, or data requirements by studying these artifacts before (or instead of) interviews. This technique helps build an initial model: the BA might draft a workflow diagram based on a procedure manual, then validate it with users. However, beware outdated docs – always confirm assumptions with stakeholders ¹⁰. Document analysis accelerates modeling by providing baseline information (e.g. regulatory requirements or form fields) that can feed directly into structured requirement lists or process maps.

Comparison and Modeling Support

Technique	Insights Yielded	Modeling Output
Interviews	User needs, steps of a process, exceptions	User stories, detailed process flow steps, use case narratives
Workshops	Group consensus on end-to-end process, brainstorming results	Collaborative process maps, swimlane diagrams, prioritization lists
Surveys/ Questionnaires	Quantitative ratings, preferences, common issues	Statistical summary to highlight focus areas (e.g. frequent errors)
Observation	Actual work flow, environment context, timing, bottlenecks	Accurate “as-is” process flows, work-arounds, condition branches
Doc Analysis	Existing rules, procedures, data definitions	Draft processes, data models, requirement lists (from policies/forms)

Each technique complements the others. In practice, analysts blend them: e.g. review policies (doc analysis), interview a few users, then run a workshop to fill gaps. The mix ensures that the structured process documentation (flowcharts, BPMN models, feature specifications) is grounded in real stakeholder knowledge and validated facts.

Characteristics of High-Quality Requirements

A requirement's quality greatly affects project success. Industry guides (including BABOK and IEEE) stress that **good requirements** must be **clear, atomic, testable, and traceable** ¹ ² . Key attributes include:

- **Clarity (Unambiguous)** – Each requirement should state exactly one functionality or condition. There must be no confusion about its meaning. In BABOK terms, clarity means “absence of ambiguity” ¹¹ . A clear requirement avoids jargon and is grammatically precise. For example, instead of saying “The system shall handle X well,” a clear statement would specify what “handle” means. Clarity often goes hand-in-hand with being **concise** – omitting unnecessary words ¹² .
- **Atomic** – A requirement should be a single, self-contained piece of functionality. Atomic means it can stand on its own without needing other requirements for context ¹³ . If a statement contains multiple parts (e.g. “The system shall create, edit, and delete user accounts”), it should be split into separate requirements for create, edit, and delete. Atomic requirements simplify understanding and tracing, since each requirement addresses one need only ¹ .
- **Testable** – You must be able to verify the requirement with tests. If you cannot design a test or criterion to confirm a requirement, it is not testable ¹⁴ . (The agile INVEST principle similarly emphasizes that stories be “Testable” ¹⁵ .) For example, “Search results should load quickly” is vague; a testable version would specify “within 2 seconds under normal load.” Testable requirements give a pass/fail condition so that QA can check them, and stakeholders know when each requirement is done.
- **Traceable** – Every requirement should be linked to its origin (e.g. business goal or stakeholder need) and its downstream design or test artifacts ² . Traceability means we can follow each requirement “back to the objectives” and forward into implementation. For instance, a “New user training module” requirement might trace back to an organizational goal of “Improve staff onboarding” and forward to design documents for that module. Traceability is often managed with a requirements matrix or tool. It ensures nothing is lost: if a requirement changes or is removed, trace links help identify impacted areas.

In summary, well-written requirements are **clear** (unambiguous and concise), **atomic** (one need per statement), **testable**, and **traceable** ¹³ ² . This quality standard helps ensure the documented requirements are useful for modeling, development, and validation.

Validation Techniques

Collecting requirements is only half the battle; we must **validate** them to ensure accuracy and completeness. Common validation techniques include:

- **Peer Reviews / Walkthroughs.** Gather a small group of colleagues or subject-matter experts to read the documented requirements or process model together. They scrutinize each requirement for missing information, contradictions, or ambiguity. According to requirements-engineering literature, conducting systematic reviews (sometimes called inspections or walkthroughs) detects errors early

¹⁶ . For example, after drafting an *order approval* process flow, a review session with a sales manager and a finance rep can catch misunderstandings or omitted steps.

- **Prototyping.** Build a simple mock-up or demonstration of the solution to gather feedback. This might be a paper sketch, a storyboard, or a low-code prototype. In the validation phase, showing stakeholders a prototype of the process or interface helps confirm the requirements. The prototype is tested by stakeholders, and their reactions are used to refine requirements. For instance, a quick software mock-up of a customer onboarding form can reveal if the data fields and flow make sense. As one guide notes, prototyping allows users to “experiment with the model and check if it meets their need,” surfacing feedback before full development ¹⁷ ¹⁸ .

- **Acceptance Criteria Review.** Defining acceptance criteria (often in **Gherkin** Given-When-Then format) for each requirement or user story is a validation step. Acceptance criteria are explicit conditions that must hold true for the story to be considered complete ¹⁹ . Reviewing them with stakeholders ensures the requirement’s intent is fully understood. For example, an acceptance criterion for an *incident response* notification might read:

Scenario: Incident Alert Sent

Given a critical incident is logged, *when* the incident report is saved, *then* an email notification should be sent to all on-call engineers.

If the criteria are unclear or missing scenarios, this review uncovers gaps. Acceptance criteria also become the basis for user-acceptance testing.

These validation steps should occur iteratively: after elicitation, use a review session to catch errors; refine requirements and update models; then, where possible, show a prototype or tangible artifact to users for feedback. Validating early prevents costly rework later.

Role-Specific Questioning Strategies

Different stakeholders have different perspectives. A savvy analyst tailors questions accordingly:

- **Executives / Sponsors.** With senior leaders, focus on high-level objectives, constraints, and ROI. Ask “**why**” questions about business goals: e.g. “What key problem will this solve for the business?” or “How will success be measured?” Executives care about benefits and trade-offs. For example, one BA suggests asking executives *before* a project: “What is the potential upside of solving this problem? What are the risks and costs if we don’t?” ²⁰ . Such questions reveal priorities (e.g. reduce cost by 20%) and alert you to risk tolerance. Executives might not know technical details, so keep language strategic. Example questions: “What metrics will improve if this succeeds?” or “What is our fallback plan if this fails?”
- **Frontline Staff / End Users.** These are the people doing the work day-to-day. Ask “**how**” and “**what**” questions about their routine tasks and pain points. Example: “Walk me through your daily process from start to finish.” Drill into details: “When you receive an approval request, what do you do next? What exceptions occur?” Encourage them to demonstrate or draw their steps. Frontline staff often reveal hidden workarounds (“we always email each other this form” or “if system is slow, I print and handwrite changes”). Questions to uncover gaps: “What are the most frustrating parts of this task?”, “What shortcuts have you created?”, or “What would your ideal process look like?” These answers help ensure the modeled process covers real conditions and avoids missing steps.

- **System Administrators / IT.** Technical stakeholders know the constraints and data behind the process. Ask about existing systems, data, and security. Example questions: “Which systems currently support this process? How do they exchange data?”, “Are there any regulatory or security requirements we must comply with?”, “What reports or logs must be produced?” System admins can highlight integration points and performance constraints. They will point out if a new requirement needs new hardware or if a proposed solution hits a known limitation. For instance, an admin might say, “Our current database only handles transactions up to X per day” – a constraint to note in requirements.

In all cases, adapt terminology: executives want business terms (ROI, customer satisfaction), front-line staff use domain language (invoice, claim, case), and admins use technical terms (API, database, SLA). Active listening and neutral probing (e.g. “could you explain that part again?”) are key across roles. As one analyst advises, prepare a list of targeted questions by feature or process step to guide each interview ²¹, but remain flexible to follow the conversation.

Requirements Elicitation Checklist

Use this checklist during planning and execution of elicitation:

- **Identify Stakeholders:** List all relevant groups (e.g. executives, end users, managers, IT, vendors). “Hidden” stakeholders (e.g. support staff, maintenance teams) are often missed.
- **Define Objectives:** Clarify what you need to learn (e.g. “document the current onboarding process”), and set meeting goals.
- **Choose Techniques:** Decide which methods suit each stakeholder group (interviews, workshops, etc.). Plan workshops or focus groups where collaboration is needed.
- **Prepare Materials:** Develop agendas, question lists, templates or survey forms. Create draft diagrams or models to use in interviews/workshops for walkthroughs.
- **Schedule Sessions:** Book time with stakeholders well in advance and share any pre-reading (like process flows or questionnaires).
- **Conduct Elicitation:** Facilitate interviews/workshops and observe carefully. Encourage open dialogue. Ask probing follow-up questions and “why/how” clarifiers.
- **Document Outputs:** Take detailed notes, record (where allowed), and capture any diagrams drawn. Immediately tidy up and label your notes for each session.
- **Validate and Review:** After each session, review findings with stakeholders (e.g. “This is what I heard; did I miss anything?”). Use peer reviews and check against objectives.
- **Update Artifacts:** Feed findings into the process models, requirement lists, or prototypes. Ensure traceability by linking info back to stakeholders and business goals.
- **Plan Next Steps:** Identify unanswered questions and schedule follow-ups. Iteratively refine requirements until stakeholders agree they are accurate.

This reusable checklist helps ensure elicitation is thorough and organized across projects.

Acceptance Criteria (Gherkin Examples)

Acceptance criteria define *when* a requirement is considered complete. In Gherkin's **Given-When-Then** format, they read like testable scenarios. Here are two sample criteria:

- *Feature: Onboarding a New Employee*

Scenario: Successful HR Onboarding Submission

- *Given* the HR specialist has filled out the new-employee form with all required fields,
- *When* they click **Submit**,
- *Then* the system records the new employee data and displays a confirmation message.

- *Feature: Order-to-Cash Invoicing*

Scenario: Generate Invoice for Approved Order

- *Given* a sales order has status "Approved" and all products are in inventory,
- *When* the Finance user triggers invoice generation,
- *Then* an invoice PDF is created with the correct customer data and emailed to the customer,
- *And* the order status updates to "Invoiced".

These criteria provide concrete success/failure checks for each feature, guiding both development and validation.

Sample Dialogue Scripts

Scenario: Interviewing a Frontline Employee (Order Processing):

BA: "Describe the steps you take when a new order arrives."

Clerk: "First I log into the order system, then I check stock availability. If items are in stock, I allocate them and schedule shipment. If not, I send a delay notice to the customer."

BA: "What happens if the system shows insufficient stock?"

Clerk: "I switch to our inventory spreadsheet to see if there's unreported stock. If I find it there, I update the system. Otherwise, I escalate to purchasing."

BA: "Who gets notified after you allocate stock?"

Clerk: "The shipping department. I usually email them and print a pick list."

(The BA notes a possible process gap: reliance on spreadsheets and manual emails.)

Scenario: Interviewing a Manager (Executive Perspective on a Project):

BA: "Our goal is to improve the customer onboarding process. How will we know we're successful?"

Manager: "We need to reduce the time from signup to full account setup. Let's say from 10 days down to 3 days."

BA: "What business objective drives that?"

Manager: "It's part of our strategy to increase customer retention. If we onboard quickly, customers stay happier."

BA: "What risks should we watch for?"

Manager: “We must maintain data accuracy. Faster processing should never compromise compliance with regulations.”

(The BA captures a clear metric (10→3 days) and a key constraint (data accuracy/regulations).)

These dialogues illustrate how tailored questions elicit useful detail: operational steps from staff, and strategic goals from leadership.

Sources

This framework combines industry-recognized practices and guides. Techniques and definitions are based on BA standards (e.g. BABOK) and practitioners’ advice ¹ ² ¹⁸ ⁵, adapted for a cross-industry audience.

¹ ¹¹ Quality Solution Requirements - A Business Analysis Blog

<https://www.bizanalysis.org/en/quality-solution-requirements/>

² ¹² ¹³ ¹⁴ ¹⁵ Requirements quality attributes examined

<https://www.scopemaster.com/blog/requirements-quality-attributes/>

³ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ The Top 5 Elicitation Techniques Used By Business Analysts

<https://www.bridging-the-gap.com/elicitation-techniques-business-analysts/>

⁴ ²¹ What Questions Do I Ask During Requirements Elicitation?

<https://www.bridging-the-gap.com/what-questions-do-i-ask-during-requirements-elicitation/>

¹⁶ ¹⁷ Requirements Validation Techniques - Software Engineering - GeeksforGeeks

<https://www.geeksforgeeks.org/software-engineering/software-engineering-requirements-validation-techniques/>

¹⁸ What is Requirements Gathering? - Jama Software

<https://www.jamasoftware.com/requirements-management-guide/requirements-gathering-and-management-processes/what-is-requirements-gathering/>

¹⁹ Acceptance Criteria: Purposes, Types, Examples and Best Prac

<https://www.altexsoft.com/blog/acceptance-criteria-purposes-formats-and-best-practices/>

²⁰ 5 Questions to Ask Before Starting a Technology Project

<https://www.bridging-the-gap.com/5-questions-to-ask-before-starting-any-technology-project/>